Rapport de conduite de projet Data & ML

Customer Feedback Intelligence

De retours clients non structurés à des irritants quantifiés et investigables

Projet personnel de portfolio • Version 2 — octobre 2026

Ce rapport décrit un prototype indépendant, nourri par une expérience antérieure en Product Management et Customer Care. Il ne correspond pas à un projet réalisé pour un ancien employeur et ne contient pas de données internes.

1. Contexte et analyse des besoins

1.1 Le point de départ

Au sein des services client, les retours arrivent rarement sous une forme directement exploitable.

Les sources sont souvent mixtes :

- commentaires libres
- tickets rédigés par un agent suite à un appel
- échanges par email
- chat
- enquêtes de satisfaction.

Leur volume rend la lecture difficile. Et les catégories de contact souvent crées par les entreprises ne correspondent pas toujours le problème réel : certains contacts sont classés « autre », d'autres sont mal qualifiés par le client, ou plusieurs irritants apparaissent dans un même message.

J'ai choisi de partir de ce problème concret plutôt que d'un modèle à déployer. Pour un Product Manager, l'enjeu n'est pas seulement de compter des mots négatifs. Il faut pouvoir répondre à trois questions :

1. quels problèmes reviennent ?
2. dans quelles proportions ?
3. quels verbatims ou éléments opérationnels permettent de les comprendre ?

> **Problématique :** *comment exploiter automatiquement des feedbacks clients afin d'identifier et quantifier de manière fiable les principaux irritants, malgré les erreurs de qualification, le volume des interactions et la diversité des sources ?*

1.2 Contexte organisationnel

Le secteur de référence est le e-commerce, avec un usage cible côté Produit et Customer Experience.

L'hypothèse de départ est qu'une organisation dispose déjà de retours clients et, idéalement, de données de commandes ou de livraison, mais que leur rapprochement avec les décisions Produit reste partiel.

1.3 Recueil du besoin et parties prenantes

Le cadrage s'appuie sur une expérience professionnelle passée, reformulée sans informations confidentielles, et sur une réflexion documentaire. Je n'ai pas mené d'entretiens avec des utilisateurs pour ce projet. Cette limite est importante : les besoins sont plausibles et structurés, mais pas encore validés auprès d'une équipe pilote.

Acteur cible	Attente principale	Contribution au pilote
PM / PO expérience client	Repérer et investiguer les irritants	Valider les usages et les décisions
Customer Care / CX	Relier catégories et réalité des contacts	Affiner la taxonomie et relire les cas
Data / ML	Garantir qualité et évaluation	Construire et mesurer le pipeline
Engineering / BI	Rendre le service fiable et consultable	Intégration, restitution, exploitation
UX / conformité	Éviter les angles morts et les usages risqués	Tests utilisateurs, données personnelles

1.4 Priorisation des besoins

Le POC vise à démontrer qu'il est possible de transformer des retours clients non structurés :

- 1. En irritants
- 2. Quantifiés
- 3. Contextualisés
- 4. Directement investigables

Pour cette expérimentation, j'ai choisi de travailler sur **Olist : un jeu de données public issu d'une plateforme brésilienne de commerce en ligne**. Il contient des avis clients, des notes de satisfaction et des informations relatives aux commandes et aux livraisons.

Ce choix permet de tester le POC sur des données publiques. La limite étant qu'il n'y a qu'une seule source de contact. Le POC ne permet donc pas de démontrer la centralisation de plusieurs canaux Customer Care.

La priorisation des besoins est qualitative : valeur métier estimée × faisabilité sur les données disponibles.

Je n'ai pas attribué de score(type RICE), faute volumes d'utilisateurs ni estimations d'effort crédibles.

Besoin	Valeur	Faisabilité Olist	Décision
Identifier les irritants exprimés	Élevée	Élevée	MVP
Quantifier volumes et satisfaction	Élevée	Élevée	MVP
Croiser avec la livraison	Élevée	Élevée	MVP
Retrouver les verbatims	Élevée	Élevée	MVP
Observer les tendances	Moyenne	Élevée	MVP secondaire
Centraliser plusieurs canaux	Élevée	Faible	Cible
Requalifier des tickets support	Élevée	Faible	Cible

1.5 Contraintes à intégrer

Trois contraintes structurent le projet.

- Le dataset est composé d'avis post-achat en portugais : ce n'est ni un flux omnicanal ni un échantillon représentatif de tous les clients
- Un même avis peut évoquer plusieurs problèmes
- Les données opérationnelles ne doivent pas être confondues avec une preuve de la cause du mécontentement

2. Audit de la solution Data proposée

2.1 Données et chaîne expérimentale

J'ai utilisé le jeu public Olist Brazilian E-Commerce : 99 441 commandes et 99 224 d'avis, dont près de 41 000 commentaires textuels exploitables après nettoyage (absence de note de satisfaction et de commentaire textuel).

Pour le POC, un échantillon stratifié de 2 001 commentaires a été retenu. La stratification est faire par note client, avec une random state pour rendre le tirage reproductible.

Les avis sont rapprochés des commandes avec le numéro de commande. Lajointure permet d'apporter un contexte de livraison.

Limite : impossible d'attribuer le commentaire à un article particulier lorsqu'une commande contient plusieurs produits.

Chaîne mise en œuvre :

1. sélection et nettoyage
2. rapprochement avec les commandes
3. classification multi-label
4. conservation de la prédiction brute
5. contrôle métier ciblé
6. indicateurs et tendances
7. consultation des verbatims.

Les traitements ont été menés dans Python/Jupyter, avec pandas, scikit-learn, sentence-transformers et l'API Mistral. Les sorties sont conservées sous forme de fichiers CSV, XLSX et Pickle.

2.2 Référence d'évaluation et critères

Avant de comparer les différentes approches de classification, j'ai dû définir les irritants que le système devait être capable d'identifier dans les commentaires clients :

J'ai construit une première taxonomie de 14 catégories à partir des problématiques Customer Care identifiées pendant le cadrage, des irritants courants du commerce en ligne et de l'exploration des commentaires Olist. L'objectif était de disposer de catégories suffisamment précises pour distinguer les principaux problèmes, mais également compréhensibles et exploitables par une équipe Produit.

Catégorie	Définition
`DELIVERY_LATE`	Le client indique que sa commande a été livrée en retard.
`DELIVERY_NOT_RECEIVED`	Le client indique ne pas avoir reçu sa commande ou son colis.
`ORDER_INCOMPLETE`	La commande reçue contient un ou plusieurs articles manquants.
`WRONG_PRODUCT`	Le client a reçu un produit différent de celui commandé.
`DAMAGED_DEFECTIVE_PRODUCT`	Le produit est arrivé endommagé, cassé ou défectueux.
`PRODUCT_QUALITY`	Le client signale une qualité de produit insatisfaisante, sans nécessairement mentionner un défaut précis.
`PRODUCT_NOT_AS_DESCRIBED`	Le produit ne correspond pas à sa description, à ses caractéristiques annoncées ou aux attentes explicitement liées à cette description.
`PRODUCT_UNAVAILABLE`	Le client signale une indisponibilité ou une rupture de stock.

Catégorie	Définition
`CANCEL_REFUND`	Le client exprime un problème lié à une annulation ou à un remboursement.
`RETURN_EXCHANGE`	Le client rencontre un problème dans une procédure de retour ou d'échange.
`CUSTOMER_SUPPORT`	Le client signale une difficulté avec le service client : absence de réponse, mauvaise prise en charge, etc.
`PACKAGING`	Le client signale un problème d'emballage ou de protection du colis.
`OTHER_ISSUE`	Un problème est explicitement exprimé, mais ne correspond à aucune catégorie définie.
`NO_ISSUE`	Aucun problème relevant de la taxonomie n'est explicitement identifiable dans le commentaire.

A savoir que :

- Un avis peut contenir plusieurs irritants
- La note ne détermine pas la catégorie

Le jeu de référence final contient 295 exemples (après exclusion de cinq cas ambigus). Parmi eux, 197 ont été validés ou corrigés humainement ; 98 sont des annotations assistées acceptées après contrôle de cohérence. Le jeu est également enrichi de cas ciblés rares : il sert à comparer des méthodes, pas à estimer la fréquence réelle des irritants.

Pour évaluer les modèles, j'ai retenu trois métriques complémentaires :

- **Micro-F1 :** mesure la performance globale du modèle, en tenant davantage compte des catégories fréquentes.
- **Macro-F1 :** mesure la performance moyenne sur les différentes catégories, en accordant autant d'importance aux catégories rares qu'aux catégories fréquentes.
- **Exact Match :** mesure la proportion de commentaires pour lesquels le modèle retrouve exactement toutes les catégories attendues, sans ajout ni oubli.

Approche	Micro-F1	Macro-F1	Exact Match
E0 — règles lexicales	0,632	0,403	0,600
E1 — embeddings Top-1	0,151	0,162	0,125
E2 — LLM zero-shot multi-label	0,710	0,599	0,566

Résultats sur les 295 exemples de référence. Sur les 197 exemples validés/corrigés humainement : E2 Micro-F1 0,725 et Macro-F1 0,611 ; E0 0,532 et 0,300.

2.3 Lecture critique de l'audit

Approche	Points forts	Limites	Conclusion
E0 — Règles lexicales	• Faible coût de mise en œuvre et d'exécution. • Règles faciles à comprendre et à vérifier. • Exact Match légèrement supérieur à E2 sur le jeu de référence complet.	• Couverture limitée aux mots et expressions prévus dans les règles. • Difficulté à reconnaître les formulations inhabituelles ou implicites.	Baseline pertinente, mais insuffisante pour traiter la diversité des verbatims.
E1 — Embeddings (Top-1)	• Permet de comparer les commentaires aux catégories à partir de leur proximité sémantique. • Ne dépend pas uniquement de mots-clés exacts.	• Configuration Top-1 : une seule catégorie prédite par commentaire. • Peu adaptée à notre problème multi-label.	Résultats insuffisants dans cette configuration, sans remettre en cause l'intérêt des embeddings avec d'autres méthodes.
E2 — LLM zero-shot	• Meilleurs Micro-F1 et Macro-F1 des trois approches. • Meilleure prise en compte de la diversité des formulations. • Possibilité d'attribuer plusieurs catégories à un commentaire.	• Risque de faux positifs : attribution d'irritants non explicitement exprimés. • Nécessité de contrôler les résultats avant leur exploitation métier.	Approche retenue pour la classification sémantique, à compléter par des contrôles métier.

3. Solution technique cible et arbitrages

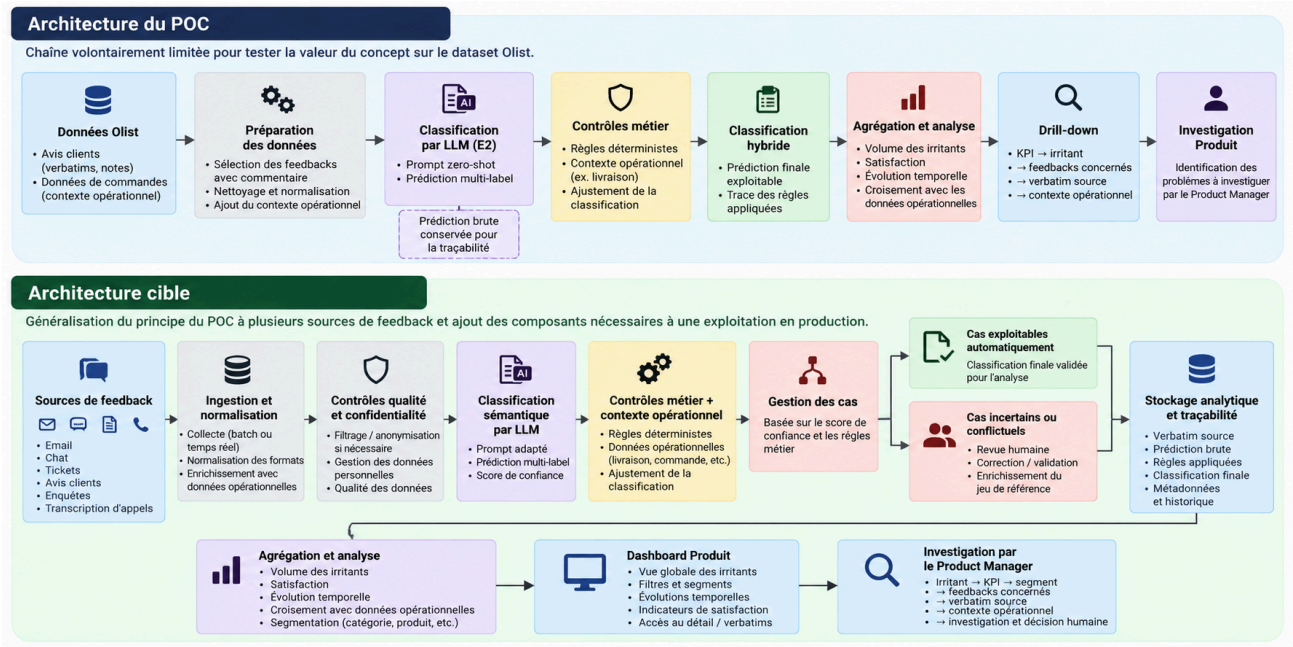
3.1 Choix de l'approche

J'ai choisi de retenir le LLM zero-shot pour analyser les commentaires, en ajoutant des contrôles pour fiabiliser ses résultats.

La solution proposée se base sur :

- **Le LLM** => identifier les irritants exprimés dans les commentaires
- **Des règles métier** => corriger certaines erreurs à partir de données vérifiables
- **Les données opérationnelles** => comprendre le contexte de certains problèmes
- **Une vérification humaine** => traiter les cas ambigus ou difficiles à interpréter

3.2 Vue d'ensemble des architectures



3.3 Cas d'usage

Cas d'usage	Question Produit	Priorité
UC1 — Regrouper	Quels irritants apparaissent dans les verbatims ?	MVP
UC2 — Quantifier	Combien de retours, quelle satisfaction, quelle évolution ?	MVP
UC3 — Contextualiser	Quel lien observable avec les données opérationnelles ?	MVP
UC4 — Investiguer	Quels messages originaux expliquent le signal ?	MVP

3.4 Principaux irritants identifiés

Sur les **2 001 commentaires analysés** :

Irritant	Avis	Part	Note moy.	Notes 1-2
Non-réception (`DELIVERY_NOT_RECEIVED`)	267	13,34 %	1,57 / 5	82,0 %
Service client (`CUSTOMER_SUPPORT`)	109	5,45 %	1,43 / 5	88,1 %

Le contrôle métier a corrigé **57 commentaires (2,85 %)** :

Catégorie	Avant contrôle	Après contrôle
`DELIVERY_LATE`	119	62
`DELIVERY_NOT_RECEIVED`	229	267

4. Appui stratégique et méthodologique

4.1 Une démarche en quatre phases

Phase	Livrables / décision	Responsable cible	Durée
1 — Cadrage, réalisé	Besoins, taxonomie, référence ; J1	Product + Data	Réalisée, non chronométrée
2 — Expérimentation, réalisée	E0/E1/E2, erreurs, arbitrage ; J2	Data + Product	Réalisée, non chronométrée
3 — Pilote, proposé	Données réelles, utilisateurs, mesures ; J3 Go/No-Go	Product, CX, Data, Engineering	À estimer après cadrage
4 — Industrialisation, proposée	Pipelines, accès, monitoring ; J4	Engineering, Data/ML, Product	Conditionnée au pilote

Les dépendances sont importantes : pas de pilote sans accord sur les données et les utilisateurs ; pas de production sans mesure de la qualité et du coût ; pas d'automatisation de décision Produit sans validation du mode d'usage. Les durées ne sont pas chiffrées faute de capacité d'équipe, de périmètre de connexion et de contraintes d'organisation connus.

4.2 Gouvernance et responsabilités

Le Product Manager porte le problème, les cas d'usage et l'arbitrage Go/No-Go.
Les équipes CX contribuent à la taxonomie et à l'interprétation.
Data/ML organise l'annotation, les expériences et le suivi de qualité.
Engineering sécurise les flux et l'exploitation.
Lles fonctions juridiques et sécurité examinent les conditions d'usage des données.

4.3 Risques, opportunités et réponses

Risque / opportunité	Conséquence possible	Mesure proposée
Biais de participation	Certains clients surreprésentés	Suivre les sources et dénominateurs
Erreurs sur catégories rares	Irritants sous-détectés	Annotation ciblée, métriques par classe
Taxonomie qui vieillit	Comparaisons instables	Versionner et documenter les changements
Dépendance LLM	Coût, latence, reproductibilité	Suivre coût/version, prévoir alternative
Données personnelles	Exposition de contenus sensibles	Minimisation, accès, revue conformité
Adoption limitée	Tableau de bord peu utilisé	Tests avec PM/CX et boucle de feedback
Opportunité : vision transverse	Investigations plus rapides	Mesurer l'usage et le temps réel gagné

4.4 Scénarios budgétaires

Une estimation monétaire sérieuse suppose de connaître le volume mensuel de messages, leur longueur, le prix réellement facturé du modèle, le taux de revue humaine et le niveau de service attendu. Ces informations n'existent pas pour le contexte cible. Je propose donc une structure de calcul et trois scénarios, plutôt qu'un montant en euros artificiellement précis.

Scénario	Hypothèses de périmètre	Principaux postes

A — POC	Une source, batch, restitution légère	Temps d'analyse, API, environnement
B — Pilote	Sources réelles limitées, utilisateurs identifiés	Connecteurs, annotations, API, dashboard, contrôle
C — Production	Multi-canal, sécurité et disponibilité renforcées	Orchestration, stockage, supervision, support

Estimation à compléter au cadrage : coût total = construction + (volume × coût moyen d'inférence) + infrastructure + supervision + (cas revus × temps moyen de revue × coût horaire) + maintenance. Le coût par feedback, le coût d'une investigation et le coût mensuel total sont les unités utiles pour comparer les scénarios.

4.5 Indicateurs de succès et décision de passage

Le pilote devra évaluer deux aspects :

1. La qualité technique

- **Performance du modèle :** Micro-F1, Macro-F1, précision et rappel par catégorie.
- **Couverture :** proportion de feedbacks pouvant être analysés.
- **Traçabilité :** possibilité de retrouver les commentaires et les corrections.
- **Coûts et performance :** coût et temps de traitement par feedback.

2. La valeur pour les utilisateurs

- Temps nécessaire pour identifier et analyser un irritant.
- Utilisation réelle de la solution par les Product Managers.

Décision de mise en production

- Les critères et les seuils de réussite seront définis avec les parties prenantes avant le pilote.
- Les performances seront évaluées sur un échantillon indépendant.
- **La mise en production dépendra à la fois de la qualité technique et de la valeur apportée aux utilisateurs.** Un bon score F1 ne suffira pas.

4.6 Impacts à anticiper

Éthique : le système rend certains problèmes visibles et d'autres moins visibles. Les résultats doivent donc rester des signaux à examiner, non un classement automatique des priorités de roadmap. Les verbatims sources doivent rester consultables, avec possibilité de correction.

Légal et confidentialité : des emails, tickets ou transcriptions réels peuvent contenir des données personnelles. Un déploiement exige une analyse des finalités, des durées de conservation, des droits d'accès, des flux vers le fournisseur LLM et des procédures applicables, avec validation par les fonctions compétentes. Le POC sur données publiques ne prouve pas la conformité de l'architecture cible.

Organisation et business : le système déplace une partie de l'effort de lecture vers la vérification et l'investigation. L'intérêt attendu est de mieux détecter et partager des signaux, mais aucun gain de temps ni ROI n'a été mesuré. Il faudra l'évaluer auprès des utilisateurs pilotes.

5. Contrôle et suivi du projet

5.1 Tableau de bord de pilotage proposé

Le pilotage sépare trois objets : la livraison du projet, la fiabilité Data/IA et la valeur Produit. Pour éviter un tableau de bord décoratif, chaque indicateur est relié à un responsable, une cadence proposée et une action possible.

Axe / indicateur	Responsable cible	Cadence proposée	Réaction si écart
Jalons, livrables, risques	PM	Hebdomadaire au pilote	Arbitrer périmètre et dépendances
Budget engagé / prévision	PM + responsable technique	À chaque revue projet	Revoir volumes et architecture
Volume, couverture, anomalies	Data / Engineering	À chaque exécution batch	Contrôler source et pipeline
F1 et erreurs par classe	Data/ML + CX	À chaque campagne annotée	Analyser erreurs et taxonomie
Latence et coût par feedback	Engineering / Data	À chaque lot, bilan périodique	Optimiser ou limiter les traitements
Usage et temps d'investigation	Product	À chaque revue utilisateurs	Adapter restitution et workflow

Cadences indicatives pour un pilote : elles ne décrivent pas une organisation existante. Aucun seuil d'alerte chiffré n'est fixé sans historique de fonctionnement.

5.2 Traçabilité des expérimentations et collaboration

Le POC conserve des notebooks, des prédictions et des fichiers de référence. Pour le pilote, chaque expérience devra documenter : date, version du dataset et de la taxonomie, méthode, modèle/version disponible, prompt, paramètres, métriques, erreurs analysées et décision. Les versions du code, des prompts, des règles et de la référence doivent pouvoir évoluer séparément.

Les besoins d'outillage sont les suivants : dépôt de code versionné, suivi des expériences, orchestration, journalisation des traitements, tableau de bord, documentation des décisions et gestion collaborative du backlog. Le choix d'une plateforme précise dépendra de l'environnement de l'organisation ; je ne présente donc pas de stack de production comme déjà déployée.

6. Conclusion et recommandations

Le POC a permis de démontrer la faisabilité d'une solution capable de :

- Identifier plusieurs irritants dans un même commentaire client.
- Quantifier les problèmes détectés.
- Croiser les résultats avec les données opérationnelles de livraison.
- Retrouver les commentaires d'origine pour faciliter l'investigation.

Enseignements

- **Meilleure approche testée :** le LLM zero-shot obtient les meilleurs scores Micro-F1 et Macro-F1.
- **Limite :** certaines erreurs de classification peuvent fausser l'analyse des problèmes clients.
- **Solution retenue :** une architecture hybride combinant LLM, règles métier et données opérationnelles, avec conservation des prédictions et des corrections.

- **Point à valider :** l'amélioration réelle de la classification grâce aux règles métier reste à mesurer sur un jeu de test indépendant.

Next step : lancer un pilote limité

- Valider les besoins avec de vrais utilisateurs PM/CX.
- Tester la solution sur des données représentatives.
- Corriger et fiabiliser les règles liées aux délais de livraison.
- Enrichir et consolider le jeu de référence.
- Mesurer la qualité des résultats et leur utilité pour les équipes Produi

7. Annexes et éléments de preuve

Dépôt GitHub du projet :
https://github.com/aurelebhm-hash/customer-feedback-intelligence
Lien fourni par l?auteur ; le contenu du dépôt n?a pas été vérifié pour cette mise à jour.

Fichiers produits dans le projet local : data/reference/e2_zero_shot_predictions.csv ; data/reference/customer_feedback_poc_predictions.csv ; data/processed/customer_feedback_poc_irritants.csv ; data/processed/customer_feedback_poc_kpis.csv ; data/processed/customer_feedback_poc_trend_validation.csv ; data/processed/customer_feedback_poc_analysis.pkl.

Sources de cadrage : template officiel de rapport de conduite de projet Data & ML ; documentation du jeu de données public Olist ; protocole et résultats expérimentaux du projet. Les hypothèses de besoin et d’organisation restent identifiées comme telles.